

UNIVERSIDAD POLITECNICA DE SAN LUIS POTOSI



Ingeniería en tecnologías de la información

Actividad 11 ROAD TO HALL OF FAME

LAB: FILE PATH TRAVERSALVALIDATION OF START OF PATH

Alumno: Isaac Elihu Rios de la Tejera

Docente: Servando López Contreras

Asignatura: CNO IV Seguridad Informática

Fecha de entrega: 26/09/2026

1. Nombre del laboratorio

File path traversal, validation of start of path

2. Nivel

Apprentice (Aprendiz)

3. Tipo de explotación

Path Traversal / Directory Traversal (Salto de directorios).

La explotación se realiza manipulando una ruta de archivo controlada mediante el parámetro filename, manteniendo el prefijo requerido por la aplicación pero incorporando secuencias ../ para salir del directorio autorizado.

4. Objetivo del laboratorio

El objetivo del laboratorio es explotar una vulnerabilidad de **Path Traversal** presente en el parámetro:

filename

utilizado por la aplicación para recuperar imágenes almacenadas en el servidor.

La finalidad es modificar la ruta solicitada para acceder a un archivo situado fuera del directorio permitido y obtener el contenido de:

/etc/passwd

Aunque la aplicación comprueba que la ruta proporcionada comience con:

/var/www/images/

esta validación resulta insuficiente porque permite incluir posteriormente secuencias de recorrido de directorios como:

../

5. Explicación técnica de la vulnerabilidad

La aplicación utiliza el parámetro filename para determinar qué archivo debe recuperar del sistema.

Una solicitud legítima puede tener la siguiente estructura:

GET /image?filename=/var/www/images/34.jpg

El servidor aplica una validación para comprobar que la ruta recibida comienza con el directorio autorizado:

`/var/www/images/`

El problema es que esta comprobación solo valida el **inicio de la cadena**, pero no verifica de manera segura la ruta final que será utilizada por el sistema de archivos.

Por ejemplo, es posible enviar:

`/var/www/images/../../../../etc/passwd`

Esta cadena continúa comenzando correctamente con:

`/var/www/images/`

por lo que puede superar la validación inicial.

Sin embargo, las secuencias:

`../`

tienen un significado especial dentro de los sistemas de archivos: indican que debe accederse al directorio padre.

Por lo tanto, la ruta:

`/var/www/images/../../../../etc/passwd`

puede resolverse de la siguiente manera:

`/var/www/images/`

`../ → /var/www/`

`../ → /var/`

`../ → /`

Después de subir tres niveles, la ruta restante es:

`etc/passwd`

por lo que la ruta normalizada termina apuntando a:

`/etc/passwd`

La vulnerabilidad existe porque la aplicación valida la cadena proporcionada por el usuario, pero no comprueba adecuadamente la **ruta canónica o normalizada resultante** antes de acceder al archivo.

6. Procedimiento paso a paso

1. Acceder al laboratorio de **PortSwigger Web Security Academy**.
2. Iniciar **Burp Suite** y utilizar el navegador configurado para enviar el tráfico a través del proxy de Burp.

3. Dentro de la tienda del laboratorio, seleccionar uno de los productos para generar una solicitud relacionada con la carga de una imagen.

4. En **Burp Suite** → **Proxy** → **HTTP history**, localizar una solicitud similar a:

GET /image?filename=/var/www/images/34.jpg

5. Enviar esta solicitud a **Burp Repeater** utilizando la opción:

Send to Repeater

6. Desde Repeater, enviar primero la solicitud original para verificar el comportamiento normal de la aplicación.

La ruta original será similar a:

/var/www/images/34.jpg

7. Modificar el parámetro filename sustituyendo el nombre del archivo por:

/var/www/images/../../../../etc/passwd

La solicitud modificada tendrá una estructura similar a:

GET /image?filename=/var/www/images/../../../../etc/passwd

8. Presionar el botón **Send** para enviar la solicitud modificada al servidor.
9. Analizar la respuesta obtenida. Si la explotación se realiza correctamente, el servidor responderá satisfactoriamente y en el cuerpo de la respuesta aparecerá información correspondiente al archivo:

/etc/passwd

Por ejemplo:

root:x:0:0:root:/root:/bin/bash

10. Regresar al navegador del laboratorio.
11. Verificar que PortSwigger muestre el mensaje:

Congratulations, you solved the lab!

y que el estado del laboratorio cambie de:

Not solved

a:

Solved

7. Capturas de pantalla obligatorias

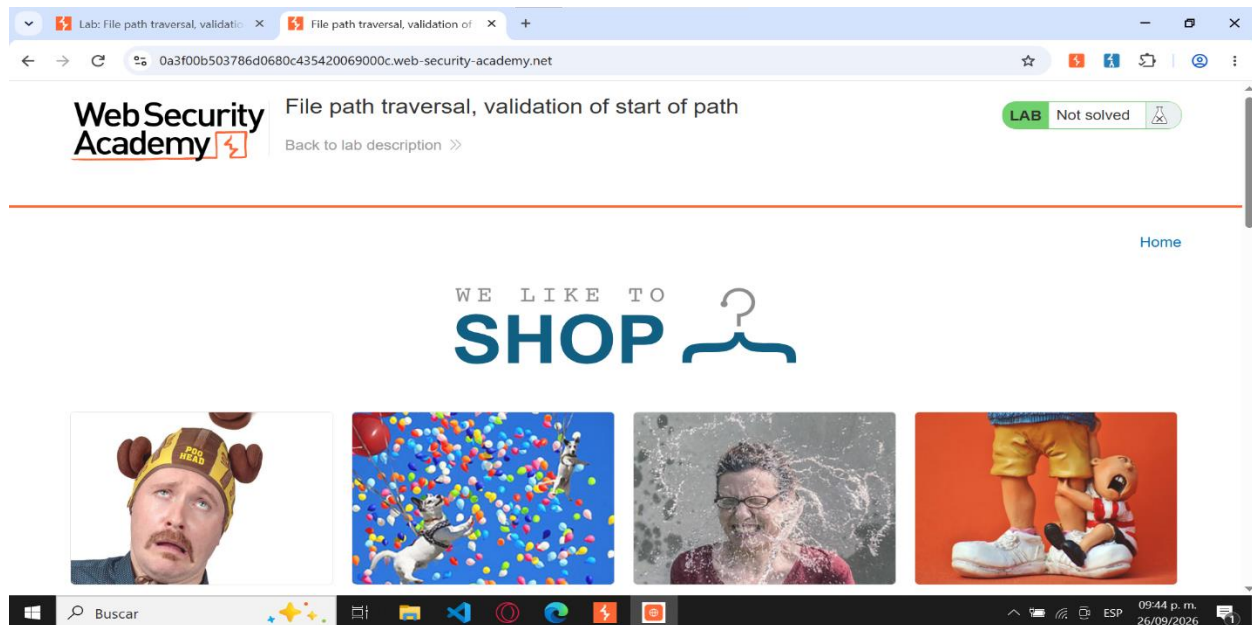


Ilustración 1 Estado inicial del laboratorio

Captura del navegador mostrando el laboratorio antes de realizar la explotación.

Debe observarse el estado:

Not solved

Esta evidencia permite demostrar el estado inicial del ejercicio antes de realizar las pruebas.

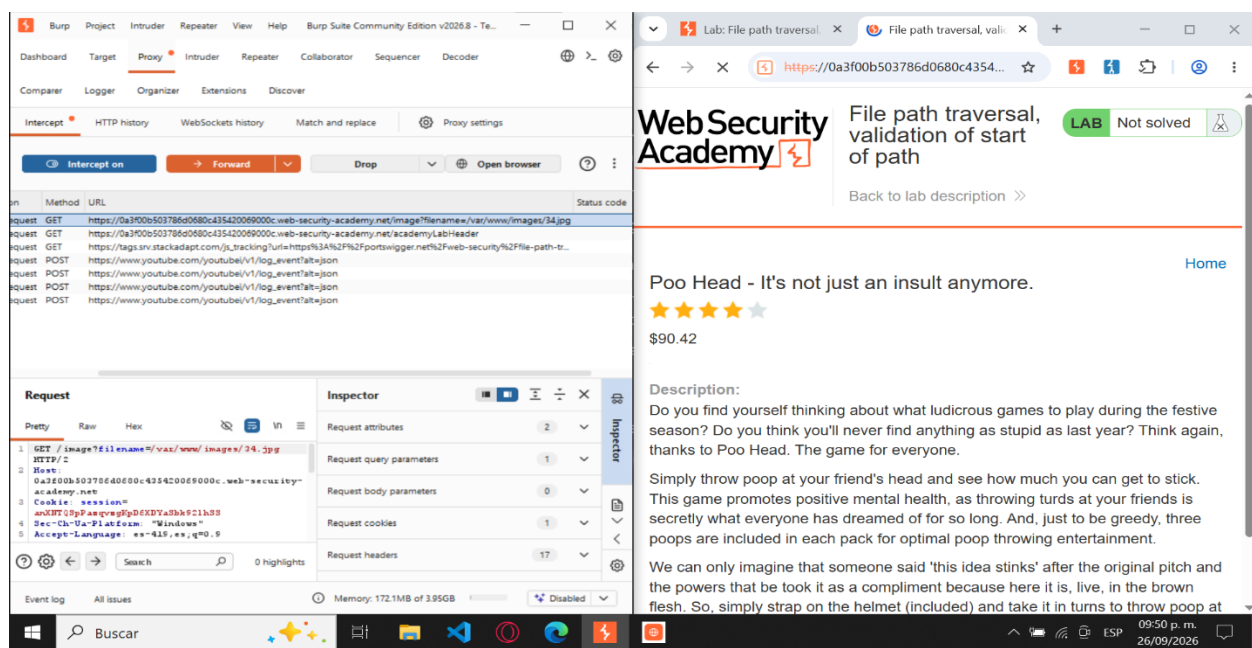


Ilustración 2 Request original interceptado

Captura de Burp Suite mostrando la solicitud legítima utilizada por la aplicación para obtener una imagen.

Por ejemplo:

GET /image?filename=/var/www/images/34.jpg

Esta captura permite identificar el parámetro vulnerable: Filename

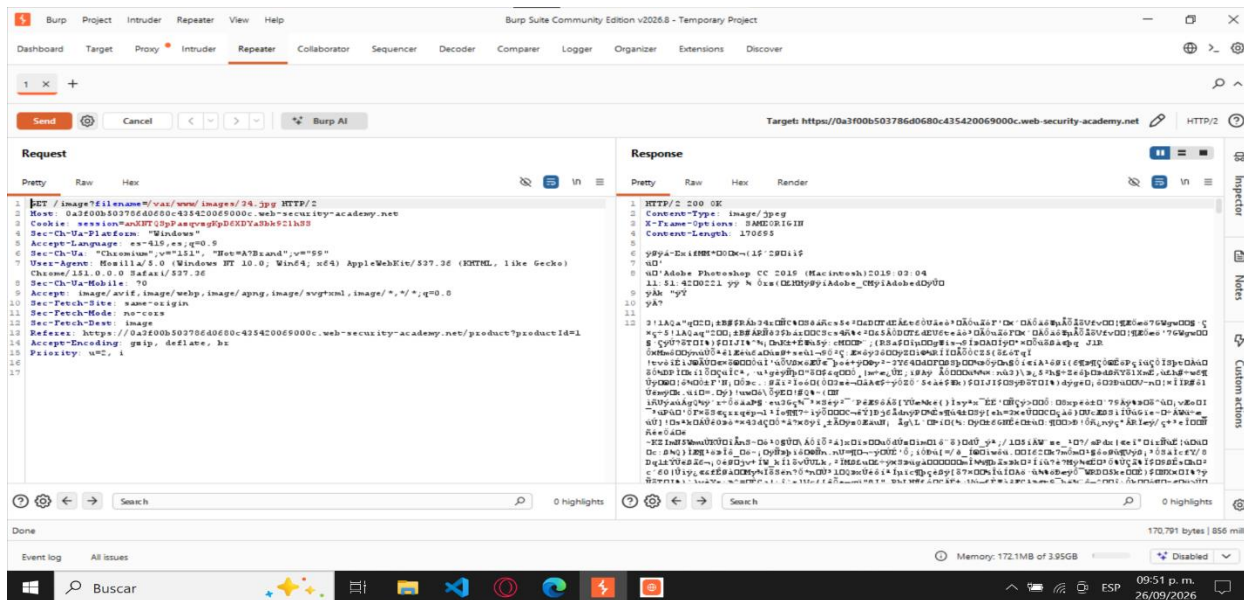


Ilustración 3 Petición original en Repeater

Captura de **Burp Repeater** mostrando la petición original antes de modificarla.

Esta evidencia es opcional, pero resulta útil para comparar el comportamiento normal de la aplicación con la solicitud manipulada.

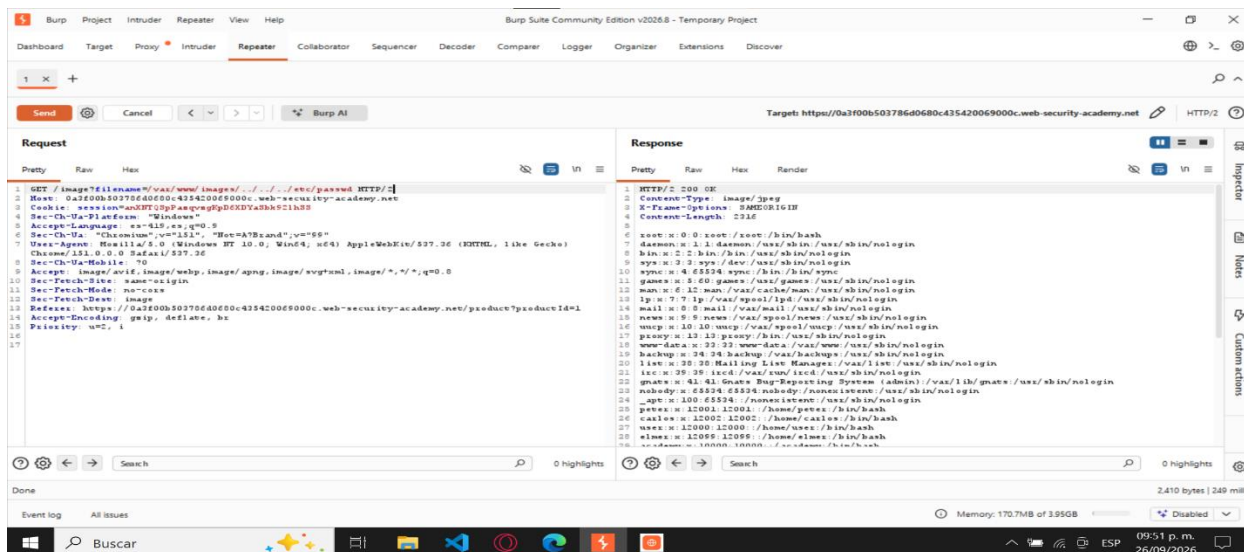


Ilustración 4 Payload utilizado y respuesta vulnerable

Captura de Burp Repeater mostrando la solicitud modificada:

```
GET /image?filename=/var/www/images/../../../../etc/passwd
```

En la misma captura debe observarse la respuesta del servidor mostrando información del archivo:

```
/etc/passwd
```

por ejemplo:

```
root:x:0:0:root:/root:/bin/bash
```

Esta es la evidencia principal de la vulnerabilidad, ya que demuestra que fue posible acceder a un archivo situado fuera del directorio autorizado.

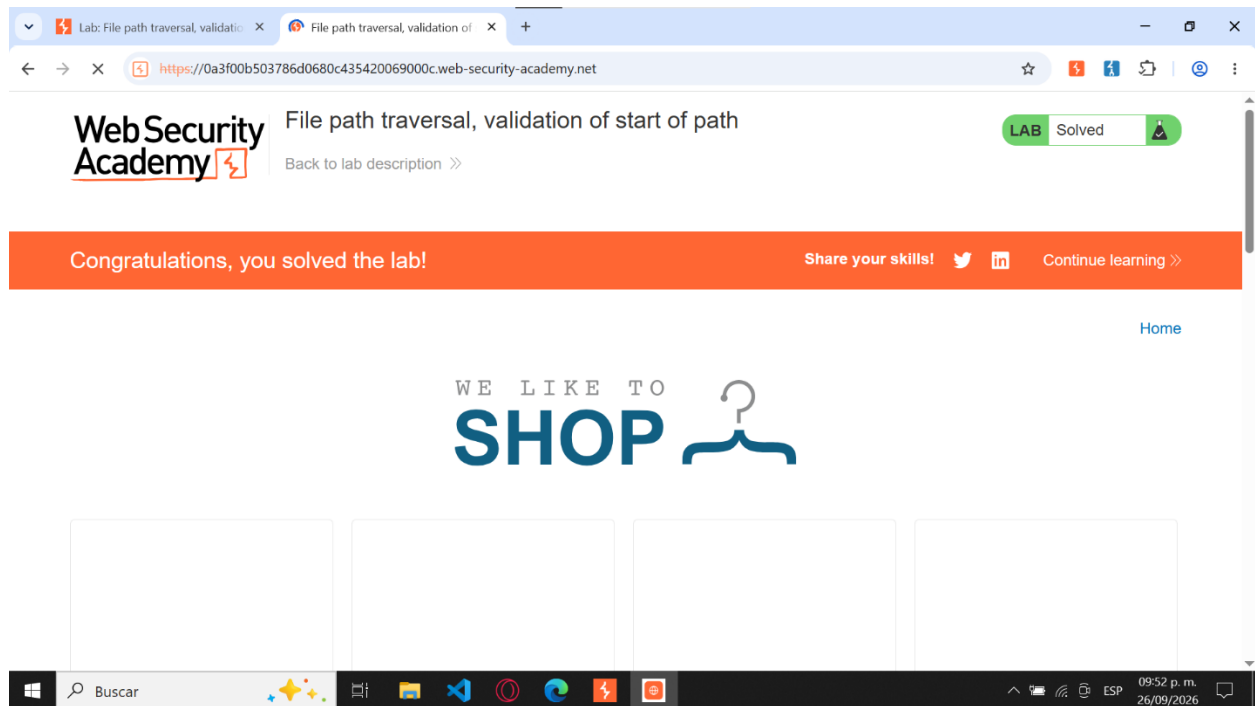


Ilustración 5 Confirmación de Lab Solved

Captura del navegador mostrando:

Congratulations, you solved the lab!

junto con el indicador:

Solved

Esta evidencia confirma que la explotación fue realizada correctamente.

8. Explicación del payload

El payload utilizado durante la explotación fue:

`/var/www/images/../../../../etc/passwd`

Puede dividirse en tres partes.

Directorio requerido por la aplicación

`/var/www/images/`

Esta sección es necesaria porque el servidor verifica que el valor del parámetro filename comience con esta ruta.

Por lo tanto, el payload conserva este prefijo para superar la validación inicial.

Secuencias de recorrido de directorios

`../../../../`

Cada secuencia:

`../`

indica al sistema de archivos que debe acceder al directorio padre.

Partiendo de:

`/var/www/images/`

el recorrido puede representarse de la siguiente manera:

`/var/www/images/`

|
├─ ../ → /var/www/
|
├─ ../ → /var/
|
└─ ../ → /

Después de las tres secuencias `../`, se alcanza el directorio raíz:

`/`

Archivo objetivo

La última parte del payload es:

`etc/passwd`

Al combinarse con el recorrido anterior, la ruta termina apuntando a:

`/etc/passwd`

Por lo tanto:

```
/var/www/images/../../../../etc/passwd
```

se normaliza conceptualmente como:

```
/etc/passwd
```

La aplicación permite la operación porque comprueba únicamente que la cadena original comience con `/var/www/images/`, pero no valida adecuadamente la ubicación final del archivo después de resolver las secuencias de recorrido.

9. Mitigación recomendada

Para prevenir vulnerabilidades de Path Traversal de este tipo se recomienda implementar diferentes controles de seguridad.

- **Evitar recibir rutas completas desde el cliente:** en lugar de utilizar valores como `/var/www/images/34.jpg`, la aplicación puede recibir únicamente un identificador, por ejemplo `id=34`, y asociarlo internamente con el archivo correspondiente.
- **Aplicar listas blancas:** aceptar únicamente archivos previamente autorizados o nombres que cumplan un formato esperado. Si únicamente se esperan imágenes, podría validarse que el nombre corresponda a los archivos disponibles.
- **Rechazar separadores y secuencias peligrosas:** entradas que contengan patrones como `../`, `..\`, `/` o `\` deberían ser rechazadas cuando no sean necesarias.
- **Normalizar o canonicalizar la ruta:** antes de acceder al archivo, el servidor debe resolver la ruta completa y comprobar su ubicación final.
- **Validar la ruta resultante:** no es suficiente comprobar que la entrada proporcionada por el usuario empiece con `/var/www/images/`. Debe verificarse que la ruta final normalizada continúe dentro del directorio autorizado.
- **Aplicar el principio de menor privilegio:** el usuario utilizado por el servidor web debe contar únicamente con permisos sobre los recursos indispensables para la aplicación.

Por ejemplo, conceptualmente el servidor debería transformar primero una ruta como:

```
/var/www/images/../../../../etc/passwd
```

a su ubicación real:

```
/etc/passwd
```

y posteriormente comprobar si esta ubicación continúa dentro de:

```
/var/www/images/
```

Como no pertenece al directorio autorizado, la solicitud debería ser rechazada.